

Enhancing Customer Churn Prediction with Integrated Feature Selection, Oversampling, and Hyperparameter Tuning

Muhammad Umer

m.umermuzaffer@gmail.com

Abstract

Customer churn causes a significant challenge to businesses, particularly in the telecom industry, where retaining existing customers is more cost-effective than acquiring new ones. This study presents a comprehensive machine learning pipeline to predict customer churn using the Telco Customer Churn dataset. The methodology integrates essential steps including data preprocessing, feature selection, oversampling, and hyperparameter tuning to enhance predictive performance. After cleaning and encoding the dataset, the class imbalance issue (74:26 ratio) was addressed using ten diverse oversampling techniques, including ROSE, ADASYN, and DistanceSMOTE. To improve learning efficiency and reduce noise, ten feature selection methods were employed, such as mutual information, ANOVA F-value, and chi-squared tests. These were systematically combined with ten classification algorithms including Logistic Regression, Random Forest, LDA, and CatBoost that yielding 1000 pipeline configurations evaluated using stratified 5-fold cross-validation. Hyperparameter tuning via GridSearchCV was applied to further optimize performance. The results demonstrate that integrating feature selection, oversampling, and tuning significantly boosts model accuracy, achieving a highest score of 80.91% with the combination of SelectFpr (p-value), MDO Oversampler, and CatBoost classifier. The proposed pipeline offers a modular and scalable solution for churn prediction and can be extended to other imbalanced binary classification problems across various domains.

Keywords: Customer Churn Prediction, Class Imbalance, Oversampling, Feature Selection, GridSearchCV, Ensemble Classifiers, Machine Learning

Introduction

Customer churn prediction is a vital analytical task for industries such as telecommunications, e-commerce, and finance, where retaining existing customers is significantly more cost effective than acquiring new ones. Churn refers to the phenomenon where customers discontinue a company's product or service. Early identification of such customers enables businesses to implement proactive retention strategies. According to a report by Bain & Company, increasing customer retention rates by just 5% can boost profits by 25% to 95% [1]. However, accurately

predicting churn is challenging due to the complex nature of customer behavior and the inherent imbalance between churned and non-churned cases in most datasets.

To address this, the current study proposes a robust and modular machine learning pipeline that systematically integrates **data preprocessing**, **feature selection**, **oversampling**, and **hyperparameter tuning**. The Telco Customer Churn dataset was used as a case study, which presented a notable class imbalance (approximately 74% non-churn vs. 26% churn). To mitigate this, we applied ten different oversampling techniques, ranging from classical methods such as BorderlineSMOTE, ADASYN, SMOTETomek and KMeansSMOTE to more advanced algorithms like DistanceSMOTE, V_SYNTH, ROSE, PDFOS, polynom_fit_SMOTE_star and MDO, sourced from the smote-variants library [2], [3]. These oversamplers were selected for their ability to generate meaningful minority class samples while preserving local density and class boundaries, a critical requirement in imbalanced learning scenarios [7].

Feature selection was employed to reduce dimensionality, remove noise, and enhance generalization. Ten filter-based feature selection techniques were used, including SelectKBest (with ANOVA F-value, Chi², Mutual Information), SelectFpr (False Positive Rate), SelectFdr (False Discovery Rate), SelectFwe (Family-Wise Error), GenericUnivariateSelect (F/MIC), and Variance Threshold [4]. These methods are known to improve model performance by eliminating irrelevant or redundant features, thus mitigating overfitting and enhancing interpretability.

The classification stage included ten supervised learning algorithms: Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), Gradient Boosting (GB), K-Nearest Neighbors (KNN), Naive Bayes (NB), Linear Discriminant Analysis (LDA), AdaBoost, Extra Trees, and XGBoost. All models were trained using Stratified 5-Fold Cross-Validation, which maintains class proportions across folds and prevents evaluation bias in imbalanced settings. Stratified K-Fold, widely regarded as a more reliable technique for model validation, is particularly effective when working with limited or skewed datasets [5].

To further enhance model performance, hyperparameter tuning was conducted using GridSearchCV. This method exhaustively searches across a specified parameter grid to identify optimal combinations that yield the best cross-validated performance. Extensive research has shown that tuning hyperparameters can significantly improve model accuracy, especially in ensemble and kernel-based classifiers [6].

In total, **1000 unique model configurations** were evaluated by combining 10 feature selection techniques, 10 oversampling methods, and 10 classifiers. The best-performing combination is **SelectFpr** (feature selection), **MDO** (Majority Directed Oversampling), and **CatBoost** (classifier) that achieved an accuracy of **80.91%**, highlighting the effectiveness of ensemble models and safe-level oversampling when paired with statistically robust feature selection. These findings support prior research which emphasizes that hybrid pipelines integrating preprocessing, resampling, and hyperparameter tuning outperform isolated approaches in real-world classification tasks [7].

2. Literature Review

Customer churn prediction has received considerable attention in recent years due to its strategic impact on customer retention and profitability. Numerous studies have employed machine learning techniques to anticipate churn behavior, particularly within the telecommunications industry. These models typically rely on customer demographics, service usage metrics, and billing history to identify patterns associated with churn.

One of the primary challenges in churn prediction is **class imbalance**, where the minority class (churned customers) is vastly outnumbered by the majority class (non-churned). Chawla et al. [2] introduced the widely adopted SMOTE (Synthetic Minority Over-sampling Technique), which generates synthetic instances by interpolating existing minority samples. Subsequent research expanded on SMOTE by developing variants such as BorderlineSMOTE, ADASYN, and SMOTETomek, each aiming to improve minority class representation while minimizing noise. Maciejewski and Stefanowski [3] further refined these ideas through DistanceSMOTE and NEATER, which consider the density and locality of minority regions to improve oversampling quality. He and Garcia [7] provided a comprehensive evaluation of various oversampling strategies, emphasizing the effectiveness of hybrid methods that combine resampling with ensemble learning.

Feature selection (FS) is another crucial component in churn modeling. It reduces dimensionality, enhances model interpretability, and mitigates overfitting. Guyon and Elisseeff [4] categorized FS methods into filters, wrappers, and embedded approaches, highlighting the advantages of statistical filter methods such as ANOVA F-test, Chi-squared, and mutual information for high-dimensional tabular data. Patil and Kulkarni [8] demonstrated that combining filter based FS with ensemble classifiers significantly improved classification accuracy in telecom churn datasets.

Several studies have also explored **the impact of classifier choice** on churn prediction. Logistic Regression, Decision Trees, and Support Vector Machines are commonly used due to their interpretability and efficiency [9]. More recent research has shown that **ensemble learning techniques**, including Random Forest, XGBoost, and AdaBoost, often outperform traditional models, especially when combined with class balancing and tuning mechanisms [10][11].

Cross-validation and hyperparameter tuning are essential for ensuring model generalization and stability. Kohavi [5] recommended stratified k-fold cross-validation as a reliable method for evaluating performance in imbalanced classification tasks. Bergstra and Bengio [6] demonstrated that hyperparameter optimization, including grid and random search, can significantly boost model accuracy and robustness, particularly in ensemble-based methods.

Despite these advancements, most prior studies either focus on a limited set of oversampling methods or evaluate models without systematically combining FS, OS, and tuning. This leaves room for a more comprehensive and modular pipeline that integrates these components and compares hundreds of configurations. The current study aims to fill this gap by exploring 1000

model pipelines that combine ten feature selection methods, ten oversampling techniques, and ten classifiers, fully tuned and evaluated using stratified cross-validation.

3. Dataset Description

The dataset used in this study is the **Telco Customer Churn dataset**, which is publicly available on Kaggle and originally sourced from IBM Sample Data Sets [12]. It contains information about a telecom company's customers, including their demographics, account details, service usage patterns, and whether they churned (discontinued the service) or not.

3.1 Dataset Overview

- Total instances (rows): 7,043
- Total attributes including the target variable (columns): 21
- Target variable: Churn (binary: *Yes* or *No*)

The goal is to accurately predict the value of the Churn column based on the remaining customer attributes.

3.2 Feature Types

Feature Type	Example Columns
Categorical (Binary)	Partner, Dependents, Phone Service, Paperless Billing
Categorical (multi-class)	Internet Service, Contract, Payment Method
Numerical	Tenure, Monthly Charges, Total Charges
Identifier (Dropped)	Customer ID
Target	Churn (Yes/No)

Most features are categorical, either binary or multi-class.

TotalCharges was originally read as an object (string) type and required conversion to numeric.

3.3. Exploratory Data Analysis

The figure below presents customer churn patterns across gender groups. Both male and female customers show similar churn rates, with slightly more males retained (2612 vs. 2541) and a near-identical number of churned customers (923 vs. 934). This suggests gender alone may not be a strong predictor of churn

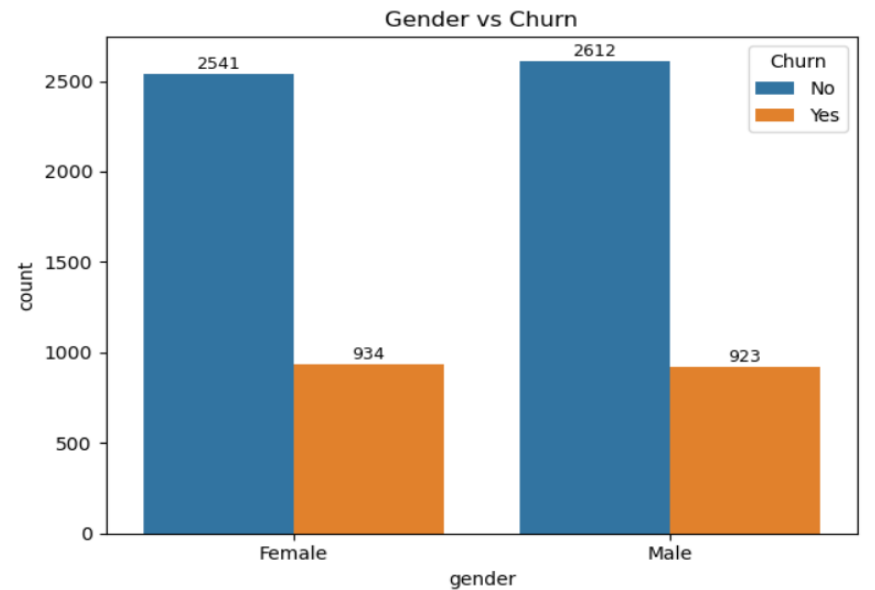


Figure below illustrates the relationship between having a partner and customer churn. Customers **without a partner** show a noticeably higher churn count (1188) compared to those **with a partner** (669), suggesting that partnership status may influence retention rates.

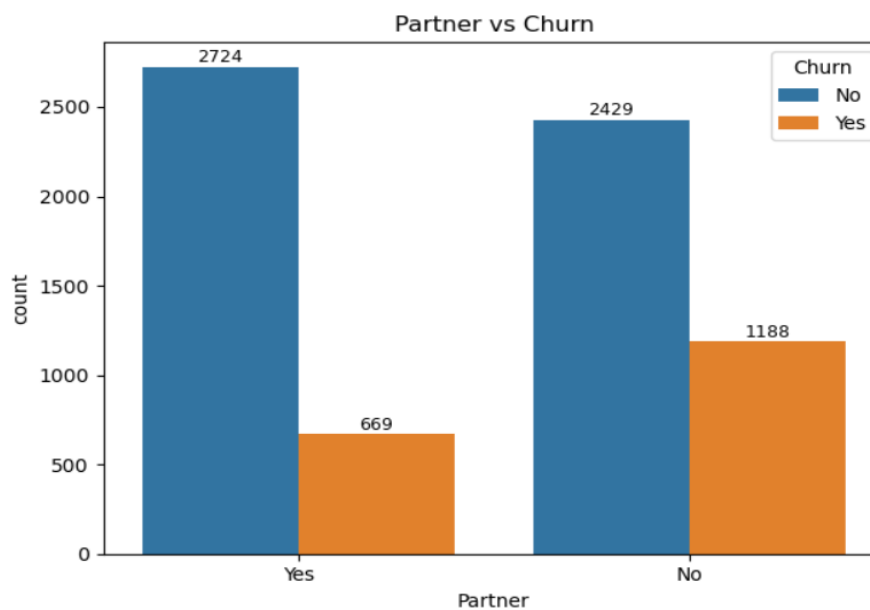


Figure below illustrates the relationship between having dependents and customer churn. Customers without dependents show a significantly higher churn count (1531) compared to

those with dependents (326), suggesting that dependent status may positively impact customer retention.

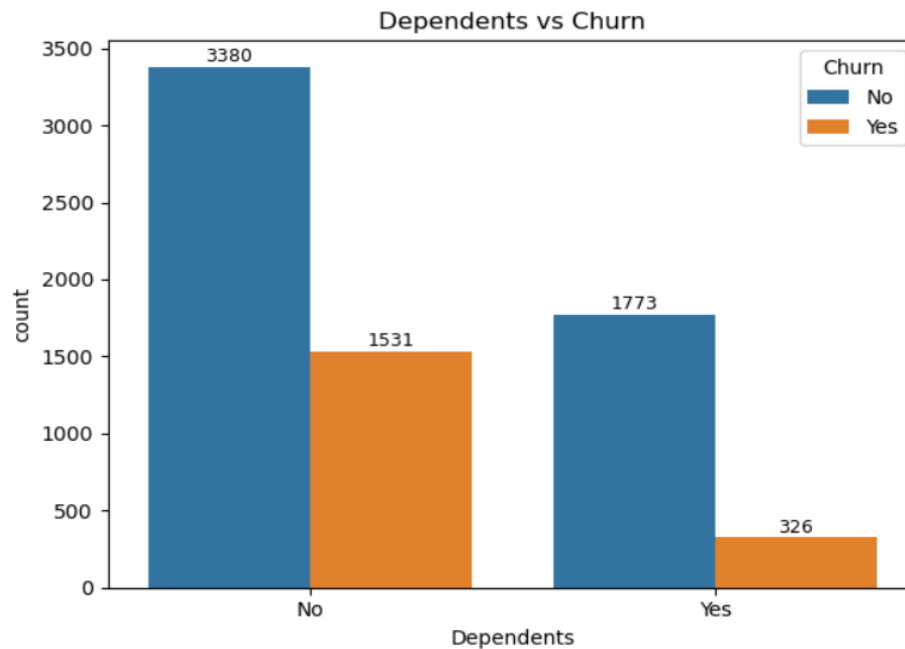


Figure below illustrates the relationship between phone service status and customer churn. Customers with phone service show a considerably higher churn count (1687) compared to those without phone service (170), likely due to their larger overall presence in the customer base. This suggests that churn dynamics may depend more on service engagement than simple service presence.

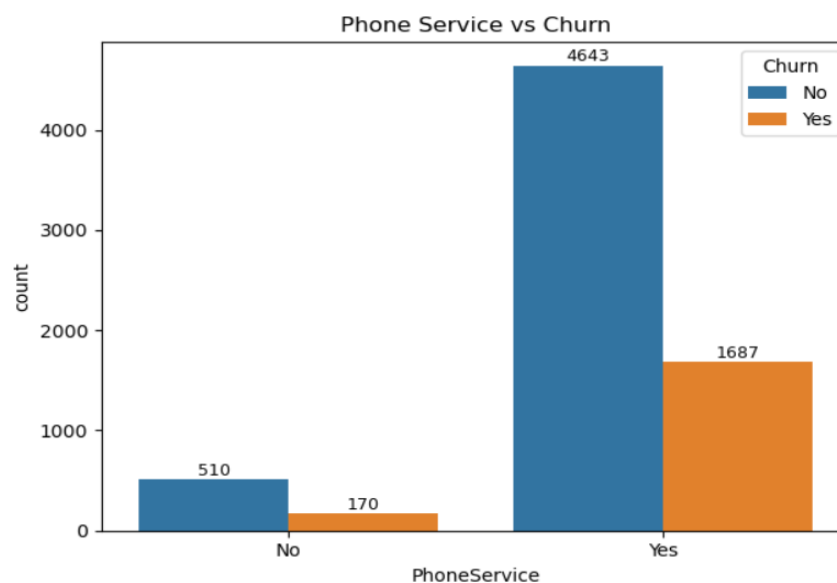


Figure below illustrates the relationship between customer tenure and churn behavior. Customers who did not churn have a noticeably higher median tenure (38 months) compared to those who churned (10 months), suggesting that longer-term customers are significantly more likely to stay. This highlights tenure as a potentially powerful predictor of retention.

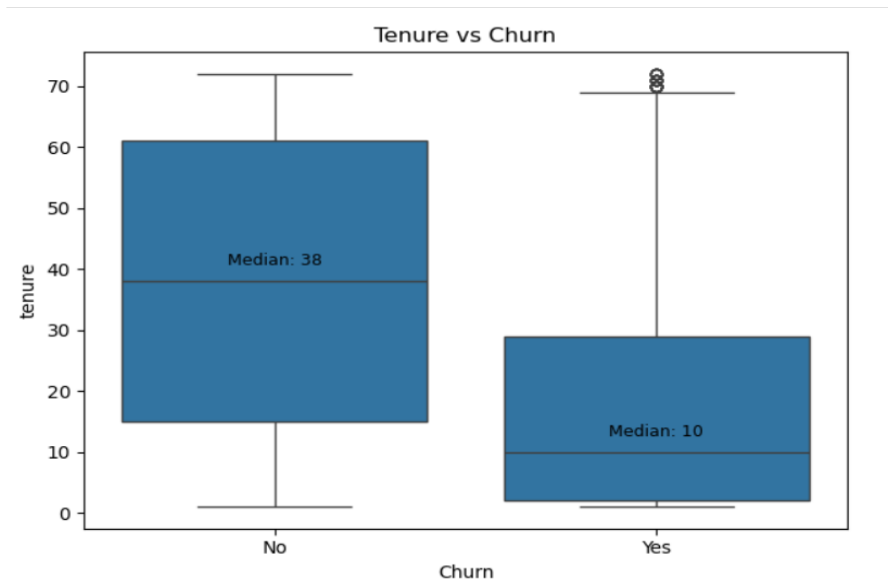


Figure below illustrates the relationship between monthly charges and customer churn. Customers who churned have a noticeably higher median monthly charge (80) compared to those who stayed (65), suggesting that higher costs may be linked to increased churn risk. This insight could inform pricing strategies or targeted retention efforts in prediction.

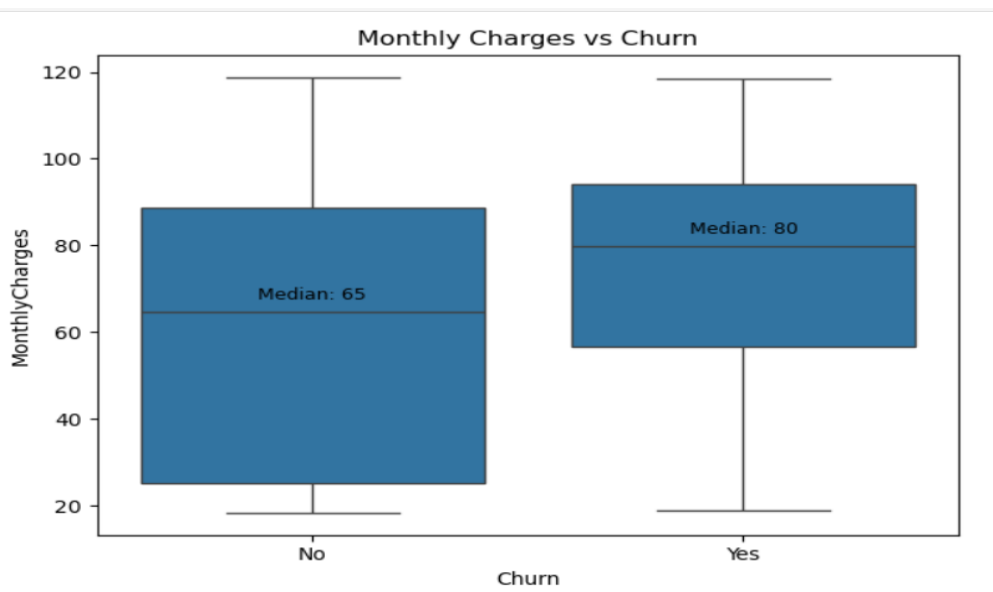


Figure below illustrates the relationship between total charges and customer churn. Customers who did not churn show a much higher median total charge (1689) compared to those who churned (713), indicating that higher accumulated spending aligns with stronger retention. This could reflect long term engagement or sustained satisfaction with the service.

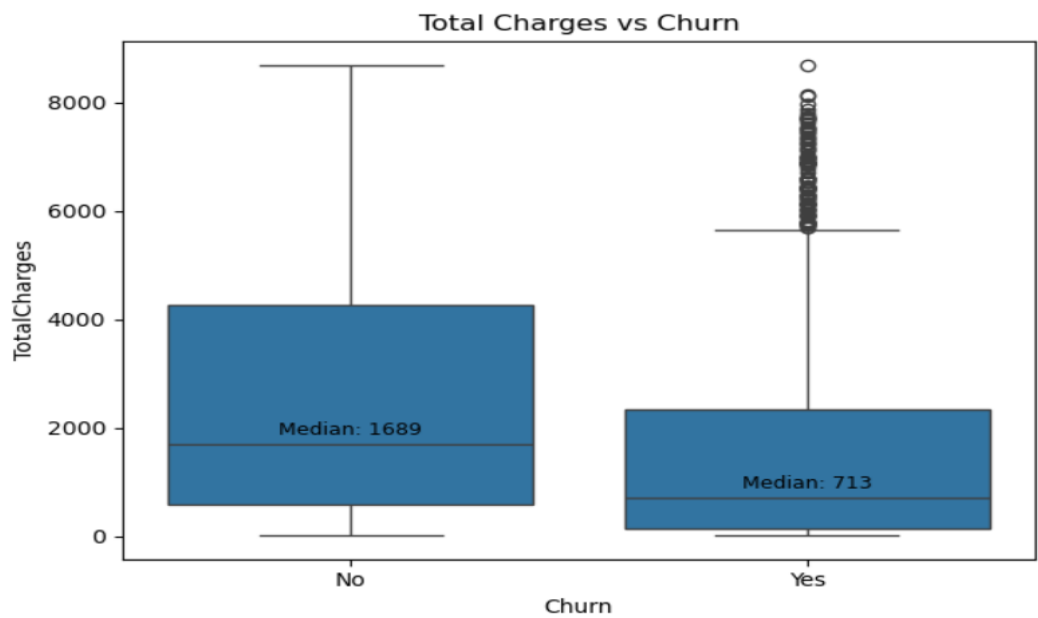


Figure below illustrates the relationship between contract type and customer churn. Customers on month-to-month contracts exhibit the highest churn rate (42.6%), while those with two-year contracts show a remarkably low churn rate (2.8%). This suggests that longer contract durations strongly correlate with greater customer retention.

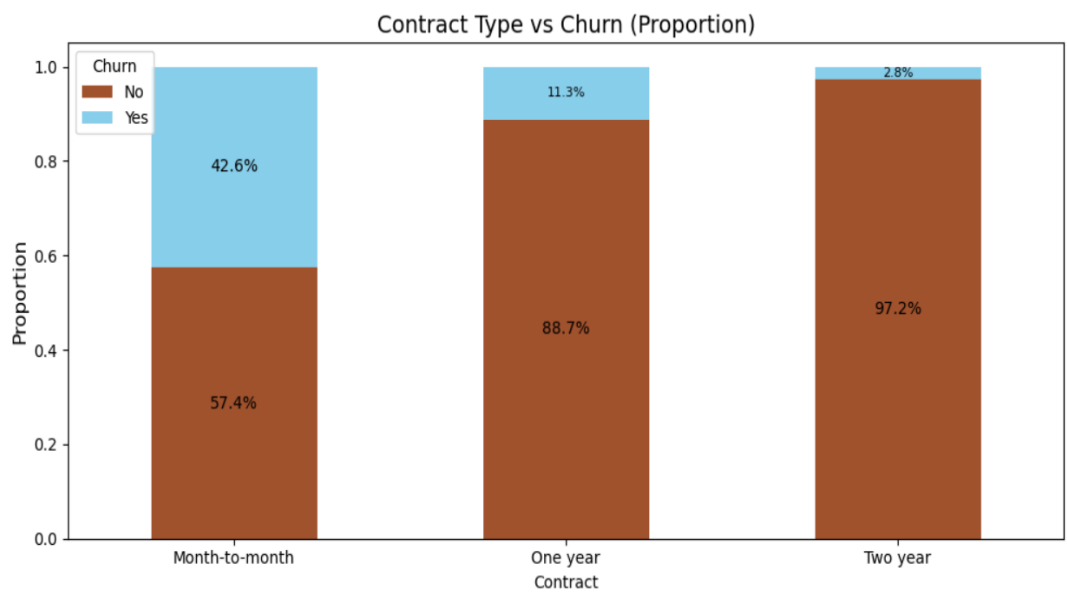
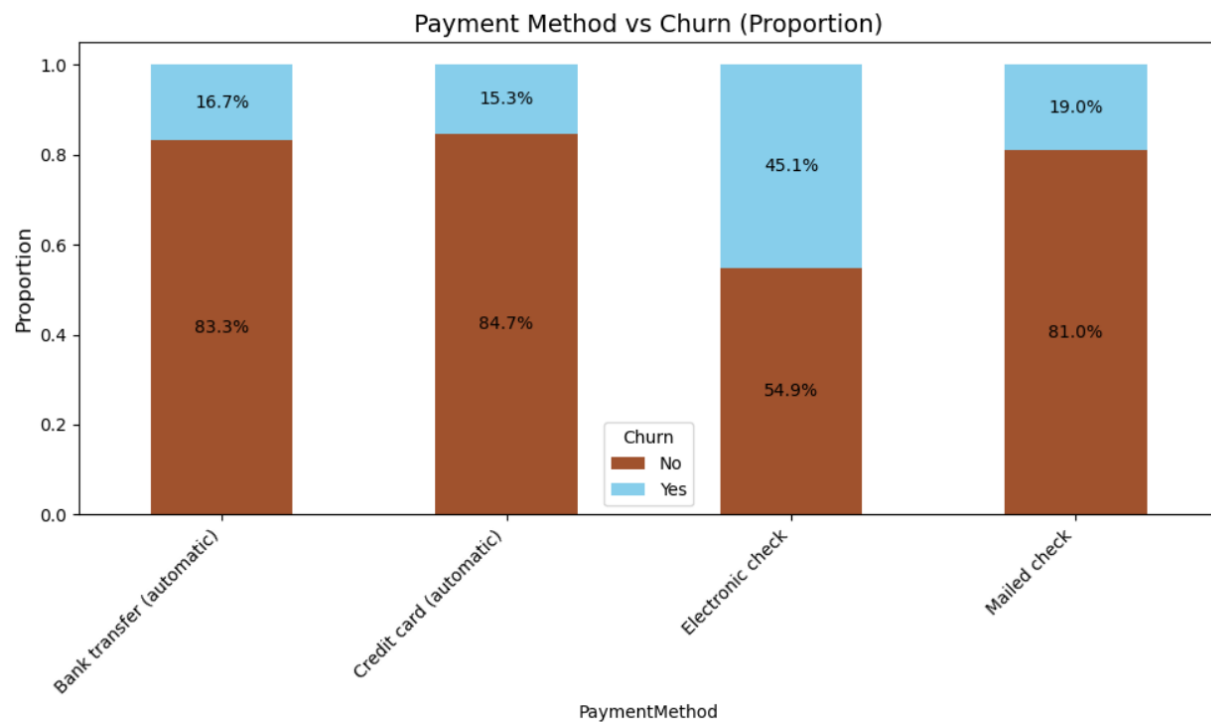


Figure below illustrates the relationship between payment method and customer churn. Customers using electronic checks exhibit the highest churn rate (45.1%), while those using automatic payments like credit cards and bank transfers show much lower churn rates (15.3% and 16.7%). This suggests that convenience and automation in payments may contribute to stronger customer retention.



3.3 Class Imbalance

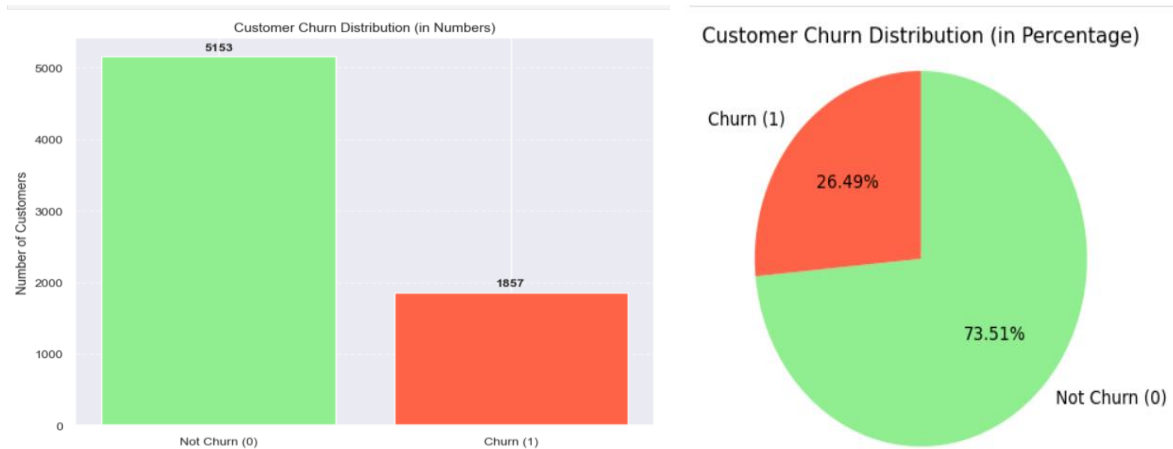
The Churn variable is highly imbalanced, which is a common challenge in real-world churn prediction tasks [13].

- No (non-churned): 5,174 customers ($\approx 73.5\%$)
- Yes (churned): 1,869 customers ($\approx 26.5\%$)

This imbalance (roughly 74:26) can bias models toward the majority class. Therefore, the dataset is an appropriate candidate for evaluating oversampling strategies [2][7].

Figures below illustrate this imbalance using a bar chart that shows the actual number of customers in each class and provides a complementary view using a pie chart that visualizes the percentage distribution between churned and non-churned customers.

These visualizations highlight the skew in the target variable, justifying the need for class balancing techniques during model training.



4. Methodology

4.1 Data Preprocessing

To ensure that the dataset was suitable for training machine learning models, several preprocessing steps were conducted. These steps were critical to handle inconsistencies, prepare categorical data for modeling, and normalize numerical features.

Dropping Irrelevant Features

The dataset contained a unique identifier column, `customerID`, which did not contribute any predictive value. This column was removed to reduce noise and focus only on meaningful variables.

Data Type Conversion

The `TotalCharges` column was stored as a text (object) type due to formatting issues and missing entries. It was converted to a numeric format to allow it to be treated as a continuous variable during model training.

Handling Missing Values

A small number of rows contained missing values after data type conversion. These rows were removed from the dataset to maintain data quality and ensure accurate model learning.

Encoding Categorical Variables

The dataset included several categorical variables in both binary (e.g., Partner, Dependents) and multi-class formats (e.g., Contract, InternetService). Binary variables were labelled encoded into 0s and 1s whereas multi-class categorical variables were converted into numerical form using one-hot encoding to eliminate any unintended ordinal relationships.

Feature Scaling

To bring numerical features onto a similar scale, standardization via **StandardScaler** was applied. This transformation adjusted the features to have zero mean and unit variance, which is particularly important for algorithms that are sensitive to feature magnitudes.

However, in specific cases such as the **Chi-squared (Chi²) feature selection method**, standardization is not appropriate because the Chi² test requires non-negative input values. For this reason, **MinMaxScaler** was applied exclusively when using Chi², scaling all feature values to a fixed range between 0 and 1. This ensured that the statistical assumptions of the test were met without compromising the integrity of the pipeline [14].

Both scaling approaches were integrated within their respective machine learning pipelines to **prevent data leakage**, a crucial consideration when applying cross-validation, especially in imbalanced datasets [15].

4.2 Feature Selection

Feature selection is a critical step in machine learning pipelines, especially when working with tabular datasets that contain a mix of categorical and numerical variables. Its primary purpose is to reduce dimensionality, eliminate irrelevant or redundant features, and improve model generalization while reducing overfitting and computational complexity [4].

As illustrated on i2tutorials [19], **Figure below** illustrates this filtering process, where the initial full feature set undergoes statistical evaluation to discard irrelevant attributes and ultimately produces a compact, high-impact subset used in model training.

In this study, ten filter-based feature selection techniques were employed. These univariate statistical methods evaluate the relationship between each input feature and the target variable independently [14].

The selection techniques used include:

- **SelectKBest** with:
 - ANOVA F-value
 - Chi-squared (Chi²)

- Mutual Information (MI)
- **GenericUnivariateSelect** using F-value and MI as scoring functions
- **SelectPercentile**
- **SelectFpr** (False Positive Rate)
- **SelectFdr** (False Discovery Rate)
- **SelectFwe** (Family-wise Error Rate)
- **Variance Threshold**

Each method has its own criterion for determining the significance of a feature:

- F-value captures linear dependency between features and labels,
- Chi-squared measures statistical independence (requiring non-negative values),
- Mutual Information detects non-linear dependencies,
- Threshold-based methods control different types of statistical error rates.

To ensure validity, MinMax scaling was applied when using Chi²-based methods, as they require non-negative input values [14]. All feature selection steps were encapsulated within their respective machine learning pipelines to avoid data leakage, ensuring that only training data influenced feature selection in each fold during cross-validation [15].

This multi-method approach allowed us to evaluate how different statistical criteria affect model performance and helped identify which combinations of selectors, oversamplers, and classifiers yielded the most accurate churn predictions.



4.3. Oversampling Techniques

The Telco Customer Churn dataset exhibited a significant class imbalance, with approximately 73.5% of customers labeled as **No** (non-churn) and only 26.5% as **Yes** (churn). This imbalance can cause classifiers to favor the majority class, often leading to poor predictive performance on the minority class and the very customers most critical to retain [2][13].

To mitigate this issue, ten oversampling techniques were employed to synthetically balance the dataset by increasing the number of minority class samples. All oversampling was applied within the training folds of the pipeline during cross-validation to ensure there was no data leakage [15].

As shown on DataAspirant [20], **Figure below** demonstrates the oversampling process where synthetic samples are generated for the minority class to balance the dataset and enhance the model's ability to learn discriminatory patterns.

Oversampling Methods Used

The oversamplers used in this study included a mix of classical techniques and advanced synthetic methods sourced from both imblearn and the smote-variants Python libraries [3][18]:

From imbalanced-learn (imblearn):

- **ADASYN** – Adaptive Synthetic Sampling that generates more data in hard-to-learn regions [2].
- **BorderlineSMOTE** – A variant of SMOTE that focuses on borderline examples between classes.
- **KMeansSMOTE** – Clusters minority instances and applies SMOTE within each cluster.
- **SMOTETomek** – A hybrid of SMOTE and Tomek Links that also removes borderline noise.

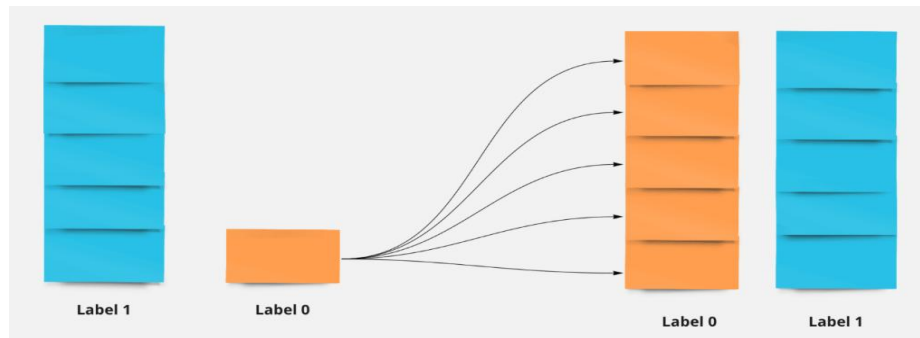
From smote variants (sv):

- **DistanceSMOTE** – Generates synthetic points based on a distance-weighted approach.
- **V_SYNTH** – Uses virtual synthesis of samples for improved generalization.
- **ROSE** – Random Over Sampling Examples; generates synthetic examples using a smoothed bootstrap.
- **PDFOS** – Probability Density Function Oversampling based on kernel density estimation.
- **MDO** – Majority Directed Oversampling that generates points influenced by the majority class geometry.

- **polynom_fit_SMOTE_star** – Fits polynomial surfaces to generate more structurally meaningful synthetic points.

Impact and Integration

Each oversampling method was applied in a modular way, integrated inside its own pipeline to preserve experimental integrity. These techniques helped rebalance the data and improve minority class prediction performance. The study systematically compared their effectiveness across multiple classifiers and feature selection techniques, ultimately identifying combinations that yielded the best results.



4.4. Classification Models

To build an accurate and robust customer churn prediction system, this study employed a wide range of ten classification algorithms. These classifiers were selected because they represent different learning strategies such as linear models, decision trees, ensemble methods, and probabilistic models are commonly used in binary classification tasks. By including this variety, we ensured that the model evaluation would cover both simple and complex decision-making processes.

All models were integrated into complete machine learning pipelines that also included data preprocessing, feature selection, oversampling, and hyperparameter tuning. The pipelines were evaluated using Stratified 5-Fold Cross-Validation, which preserved the class distribution (churned vs non-churned) in every fold. This helped provide a fair and consistent comparison of model performance across all combinations.

The following ten classifiers were used:

- **Logistic Regression:** A simple and fast linear model often used as a baseline in binary classification tasks like churn prediction [21].

- **Decision Tree:** Splits data into branches based on feature values. It is easy to understand but can be overfit without pruning [22].
- **Random Forest:** Builds many decision trees and takes their majority vote, helping reduce overfitting and improving accuracy [23].
- **Gradient Boosting:** Trains multiple trees in sequence, where each new tree tries to fix the errors of the previous one. It works well on many real-world datasets [24].
- **AdaBoost:** Focuses more on the data points that previous models got wrong. It's useful for improving weak models [25].
- **CatBoost:** A modern boosting algorithm that works well with categorical data and often performs better with fewer tuning steps [26].
- **Naive Bayes:** A simple statistical method based on probability. It works well on small datasets and text classification problems [27].
- **K-Nearest Neighbors (KNN):** Predicts based on the labels of nearby data points. It's easy to use but can be slow for large datasets [28].
- **XGBoost:** An optimized version of gradient boosting that is fast, accurate, and used in many machine learning competitions [29].
- **Linear Discriminant Analysis (LDA):** A technique that finds a line (or surface) to best separate the two classes using their means and variance [30].

Each classifier was tested across all 10 oversampling methods and 10 feature selection techniques. In total, **1,000 unique model configurations** were trained and evaluated. This systematic experimentation helped identify which classifiers work best when combined with specific preprocessing steps for imbalanced churn prediction tasks.

The diverse mix of models allowed us to assess both simple and complex algorithms in terms of their ability to generalize on the imbalanced Telco churn dataset. The results from this section laid the foundation for the next stage: fine-tuning the best-performing classifiers using hyperparameter optimization.

4.5. Hyperparameter Tuning

Hyperparameter tuning is a crucial step in machine learning that involves selecting the optimal combination of model parameters that are not learned from the training data but rather set before the learning process begins. These hyperparameters directly influence a model's ability to learn effectively, balance bias-variance tradeoffs, and generalize well on unseen data [31].

In this study, hyperparameter tuning was performed using **GridSearchCV**, a brute-force method that systematically tests every combination of the given hyperparameter values. The grid search was wrapped inside the full machine learning pipeline, which also included preprocessing, feature selection, and oversampling, ensuring that hyperparameter tuning did not introduce any data leakage.

To ensure a fair and reliable evaluation, each hyperparameter configuration was validated using Stratified 5-Fold Cross-Validation. This technique divides the training data into five folds while maintaining the original class distribution (churn vs non-churn) in each fold. The model is trained and validated across all folds, and the average performance is used to select the best hyperparameters. This approach helps in preventing overfitting and ensures stable performance on imbalanced datasets [32].

Each classifier had its own set of hyperparameters defined based on common practices in literature and real-world applications. For example, tree-based models like Random Forest and Gradient Boosting were tuned on parameters such as `n_estimators`, `max_depth`, and `learning_rate`, while linear models like Logistic Regression were optimized using `C` (regularization strength) and penalty type. For CatBoost and XGBoost, additional tuning was done for `depth`, `l2_leaf_reg`, and `iterations`.

A complete list of classifiers and their hyperparameter search spaces is shown below in **Table 1**.

The tuning process significantly improved model accuracy compared to default settings. Best-performing parameters were selected based on the highest mean cross-validation score across all five folds, and these tuned models were then used for final evaluation and comparison in the results section.

No	Classifier	Hyperparameters
1	Logistic Regression	penalty: ['l2'] C: [0.01, 0.1, 1, 10] solver: ['liblinear']
2	Decision Tree	max_depth: [5, 10, None] min_samples_split: [2, 5] min_samples_leaf: [1, 2]
3	Random Forest	n_estimators: [100, 200] max_depth: [10, 20, None] min_samples_split: [2, 5] min_samples_leaf: [1, 2]
4	Gradient Boosting	n_estimators: [100, 200] learning_rate: [0.01, 0.1] max_depth: [3, 5]
5	AdaBoost	n_estimators: [50, 100] learning_rate: [0.1, 0.5, 1.0] estimator: DecisionTreeClassifier(max_depth=1)

6	CatBoost	iterations: [100, 200] learning_rate: [0.05, 0.1] depth: [4, 6] l2_leaf_reg: [1, 3] border_count: [32, 64]
7	Naive Bayes	No hyperparameters tuned (default GaussianNB used)
8	K-Nearest Neighbors (KNN)	n_neighbors: [3, 5, 7, 9] weights: ['uniform', 'distance']
9	XGBoost	n_estimators: [50, 100] learning_rate: [0.05, 0.1] max_depth: [3, 5]
10	Linear Discriminant Analysis (LDA)	solver: ['svd', 'lsqr'] shrinkage: [None, 'auto']

5. Evaluation of Models

Evaluating model performance is essential to ensure that predictions are reliable and meaningful in real-world scenarios, especially in customer churn prediction where the cost of missing a churner can be significant. In this study, multiple evaluation metrics and a cross-validation approach were used to assess each model combination effectively.

5.1 Stratified K-Fold Cross-Validation

To ensure fair and consistent evaluation, we used Stratified 5-Fold Cross-Validation. This technique splits the dataset into five equal parts (folds) while preserving the original class distribution of churn vs. non-churn in each fold. Each model is trained and validated five times, with different folds used for validation each time. The average of these results provides a reliable estimate of the model's performance and helps prevent overfitting [33].

Figure below illustrates the k-fold cross-validation process, showing how class balance is maintained in each fold [33].

5.2 Evaluation Metrics

The following metrics were used to measure model performance, calculated from the confusion matrix:

- **Accuracy:** Overall correct predictions.

$$\text{Accuracy} = \{TP + TN\} / \{TP + TN + FP + FN\}$$

- **Precision:** How many predicted churners were actually churners.

$$\text{Precision} = \frac{\{TP\}}{\{TP + FP\}}$$

- **Recall:** How many actual churners were correctly identified.

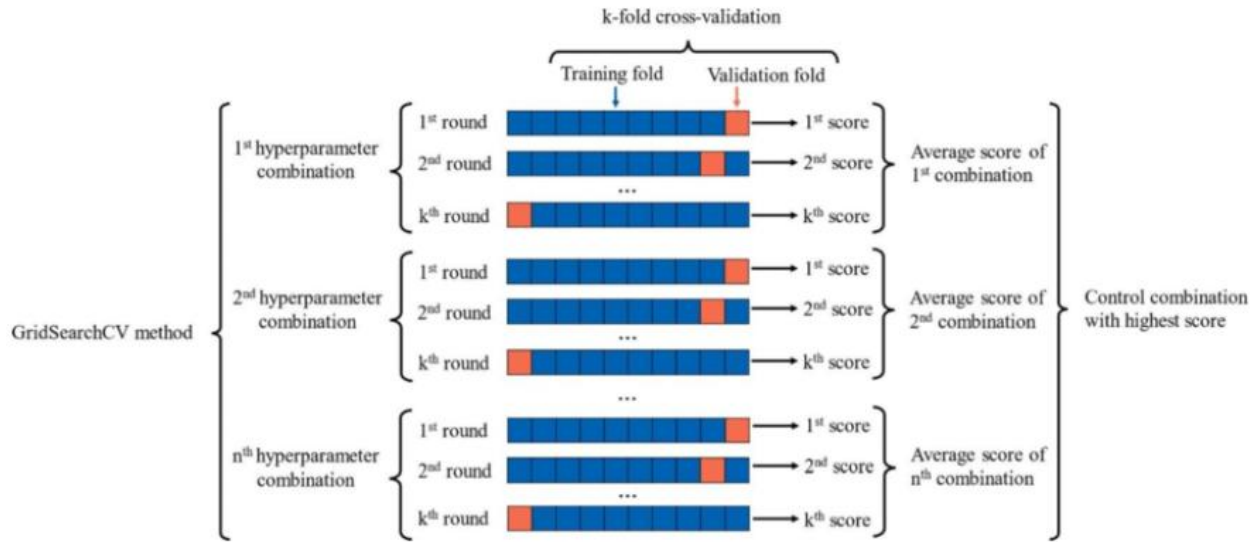
$$\text{Recall} = \frac{\{TP\}}{\{TP + FN\}}$$

- **F1-Score:** Balance between precision and recall.

$$F1 = 2 * \frac{\{Recall\} * \{Precision\}}{\{Recall\} + \{Precision\}}$$

5.3 Evaluation Protocol

Each of the 1000 pipeline combinations (feature selectors + oversamplers + classifiers) was evaluated using cross-validation. All preprocessing steps were integrated into the pipeline to prevent data leakage, ensuring fair and consistent metric calculations across all experiments.



6. Results and Discussion

This section presents the results of the machine learning experiments conducted using various combinations of feature selection methods, oversampling techniques, and classification algorithms. A total of 1000 different model pipelines were evaluated using Stratified 5-Fold Cross-Validation, and the accuracy metric was used to identify the best-performing configurations.

6.1 Overall Performance Trends

The results demonstrated that integrating feature selection and oversampling significantly improved the accuracy of most classifiers. Specifically:

- Oversampling methods like DistanceSMOTE, KMeansSMOTE, MDO, and ADASYN helped balance the dataset and improve predictive performance.
- Feature selectors such as SelectFpr (p-value), Variance Threshold, and SelectFdr (False Discovery Rate) effectively reduced irrelevant features and boosted accuracy.
- Ensemble models like CatBoost, AdaBoost, and Gradient Boosting consistently outperformed simpler models like Naive Bayes, Decision Trees, and K-Nearest Neighbors.

6.2 Best Performing Model

The best accuracy achieved among all 1000 combinations was:

- **Accuracy: 80.91%**

This result was obtained using the following configuration:

- **Feature Selection Method:** SelectFpr (p-value)
- **Oversampling Technique:** MDO
- **Classifier:** CatBoost

This combination demonstrates the effectiveness of combining a statistical filter, advanced synthetic oversampling, and a powerful gradient boosting classifier for customer churn prediction.

6.3 Classifier Performance Overview

Classifier	Observations
CatBoost →	Delivered the highest accuracy when combined with SelectFpr and MDO.
AdaBoost →	Performed well with DistanceSMOTE and variance-based feature filtering.
Gradient Boosting →	Competitive results with KMeansSMOTE and SelectFdr.
Random Forest →	Stable and strong performers with most selectors.
Logistic Regression→	Simple and interpretable, they worked best with SelectKBest and SMOTETomek.

Classifier	Observations
Naive Bayes →	Lower accuracy due to strong assumptions and feature interactions.

6.4 Implications

The results validate the importance of combining preprocessing techniques with model tuning. By experimenting with various combinations, this study demonstrates how businesses can improve churn prediction by selecting the most accurate pipeline. The 80.91% accuracy obtained offers a reliable foundation for developing targeted retention strategies.

Figure below showing the table of top 10 results by accuracy. The top performing pipeline SelectFpr with MDO and CatBoost achieved the highest accuracy of **0.8091**, slightly outperforming other combinations that hovered around 0.806–0.808. This suggests that while most configurations were competitive, targeted feature selection and domain specific oversampling gave a modest edge in overall correctness.

	Selector	Oversampler	Classifier	Accuracy	Precision	Recall	F1
0	SelectFpr (p-value)	MDO	CatBoost	0.8091	0.6773	0.5337	0.5968
1	SelectFpr (p-value)	V_SYNTH	CatBoost	0.8084	0.6792	0.5240	0.5914
2	Variance Threshold	polynom_fit_SMOTE_star	CatBoost	0.8077	0.6761	0.5256	0.5914
3	SelectFwe (Family-wise Error)	ROSE	GradientBoosting	0.8071	0.6670	0.5428	0.5980
4	Variance Threshold	ROSE	CatBoost	0.8068	0.6742	0.5245	0.5899
5	SelectFdr (False Discovery Rate)	V_SYNTH	CatBoost	0.8068	0.6762	0.5196	0.5876
6	SelectFdr (False Discovery Rate)	ROSE	CatBoost	0.8068	0.6738	0.5245	0.5896
7	SelectFpr (p-value)	polynom_fit_SMOTE_star	CatBoost	0.8067	0.6742	0.5229	0.5888
8	SelectFdr (False Discovery Rate)	polynom_fit_SMOTE_star	CatBoost	0.8067	0.6742	0.5229	0.5888
9	SelectFdr (False Discovery Rate)	MDO	CatBoost	0.8067	0.6716	0.5283	0.5911

Figure below shows a stark performance gap between the best and worst pipelines. The top performer is SelectFpr with MDO and CatBoost achieved 0.8091 accuracy due to effective feature relevance filtering and a high capacity classifier well-suited to structured features. In contrast, the poorest combination V_SYNTH with Logistic Regression underperformed (0.5275 accuracy), largely because the linear model lacked the flexibility to learn complex patterns introduced by synthetic sampling.

🏆 Best by Accuracy:	
Selector	SelectFpr (p-value)
Oversampler	MDO
Classifier	CatBoost
Accuracy	0.8091
Precision	0.6773
Recall	0.5337
F1	0.5968
BestParams	{'border_count': 64, 'depth': 4, 'iterations':...
Name	0, dtype: object
🏆 Worst by Accuracy:	
Selector	SelectFpr (p-value)
Oversampler	V_SYNTH
Classifier	LogisticRegression
Accuracy	0.5275
Precision	0.2681
Recall	0.5059
F1	0.3477
BestParams	{'C': 0.01, 'penalty': 'l2', 'solver': 'liblin...
Name	999, dtype: object

Figure below presents the confusion matrix for the top-performing pipeline (SelectFpr + MDO + CatBoost). The model identified 212 true positives and 926 true negatives, with relatively few false positives (104) and false negatives (160). This balance reflects strong discriminative power, particularly in minimizing misclassification of the majority class. The CatBoost classifier, combined with targeted feature filtering and domain-aware oversampling, enables nuanced decision boundaries even in the presence of class imbalance.

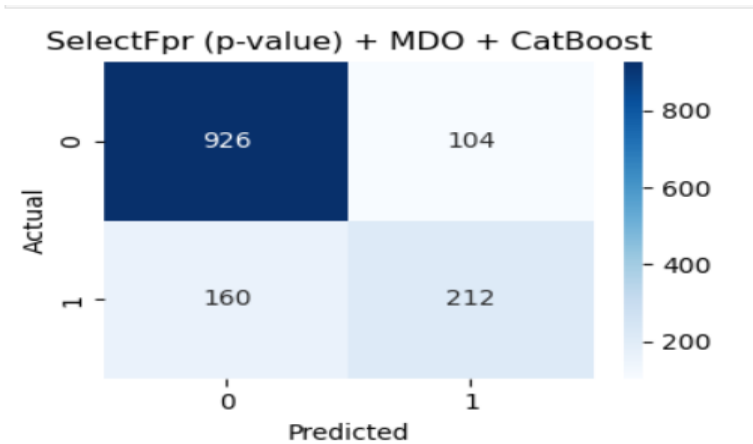


Figure below summarizes the average accuracy across classifiers. Ensemble methods like GradientBoosting and CatBoost led the field, likely due to their ability to model complex, non-linear relationships and handle skewed class distributions. In contrast, simpler algorithms such as Naive Bayes and Logistic Regression showed weaker performance, suggesting limitations in capturing the non linear relation and feature interactions present in the oversampled data.

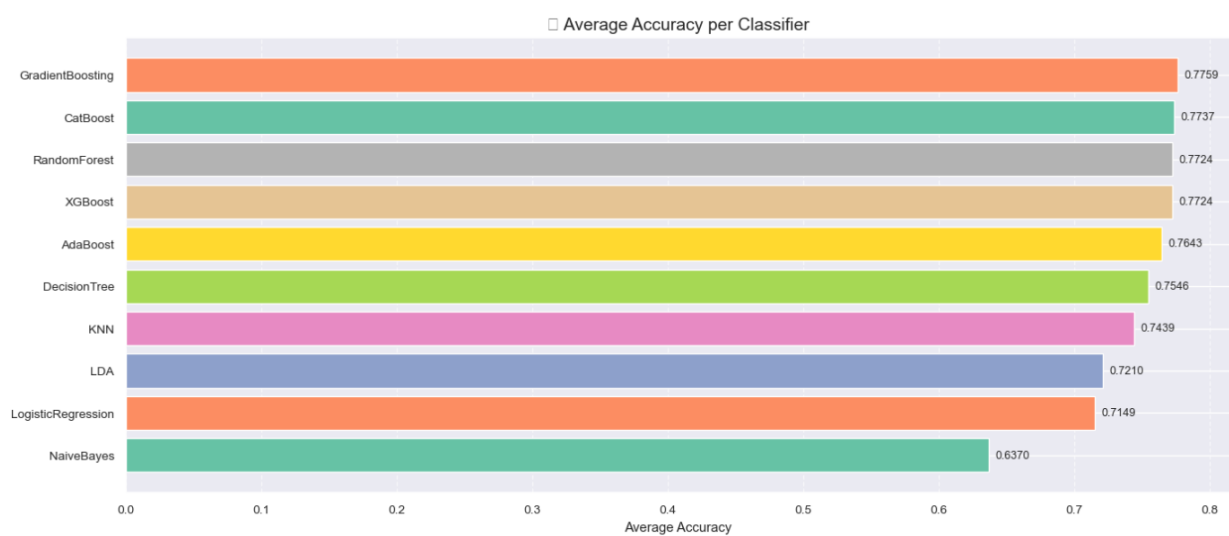


Figure below illustrates classifier accuracy across feature selection methods. CatBoost consistently emerged as the top performer across most selectors, peaking at 0.8091 accuracy with SelectFpr (p-value). This consistency highlights its adaptability to varied feature subsets. Simpler models like DecisionTree, while occasionally effective (e.g., with Chi²), generally lagged behind ensemble methods.

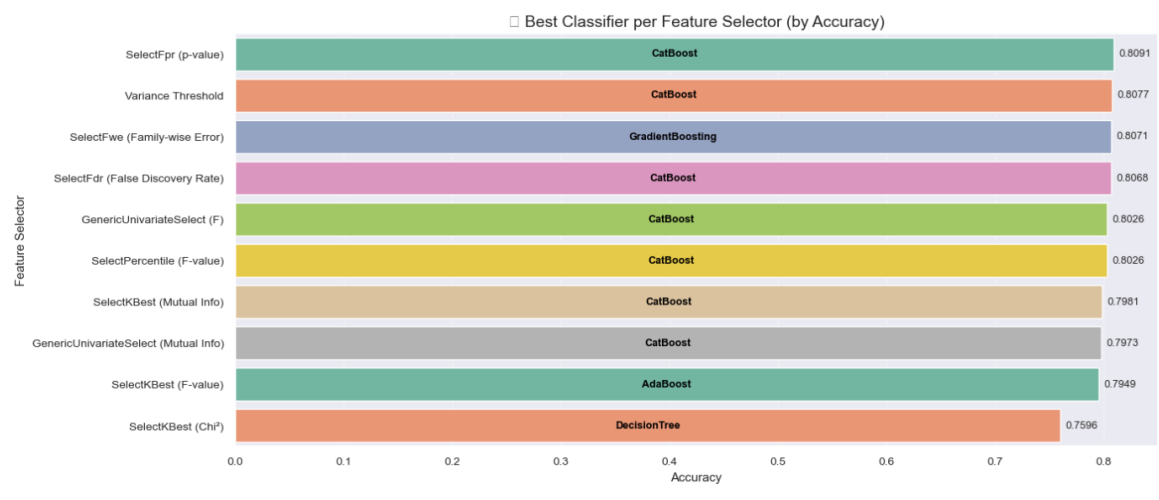
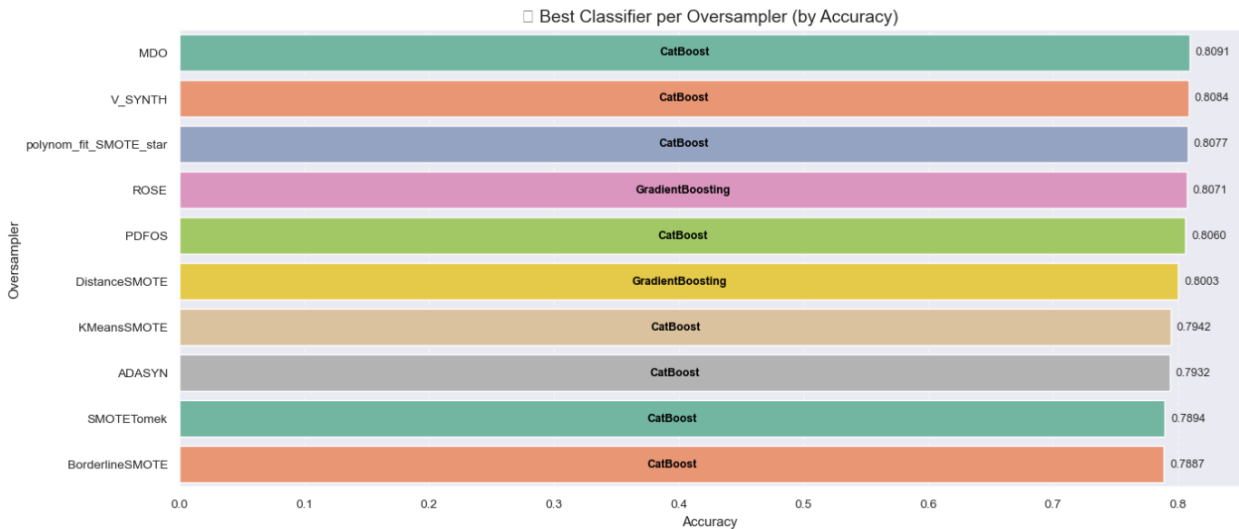
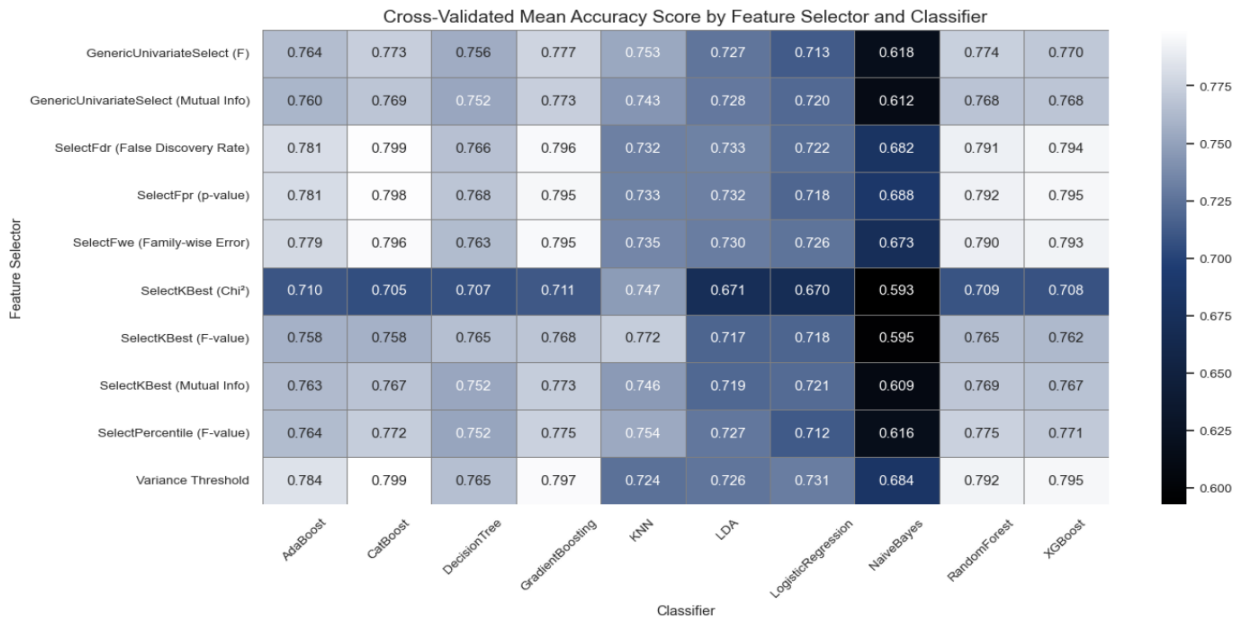


Figure below demonstrates how different oversampling techniques impact classifier performance. **MDO**, **V_SYNTH**, and **polynom_fit_SMOTE_star** led to superior accuracy when paired with CatBoost, with MDO reaching **0.8091**. These methods effectively reconstructed minority class boundaries, reducing false negatives and enhancing model generalization. In contrast, simpler strategies like BorderlineSMOTE and SMOTETomek yielded comparatively lower gains, indicating the importance of domain-aware augmentation in handling imbalanced datasets.

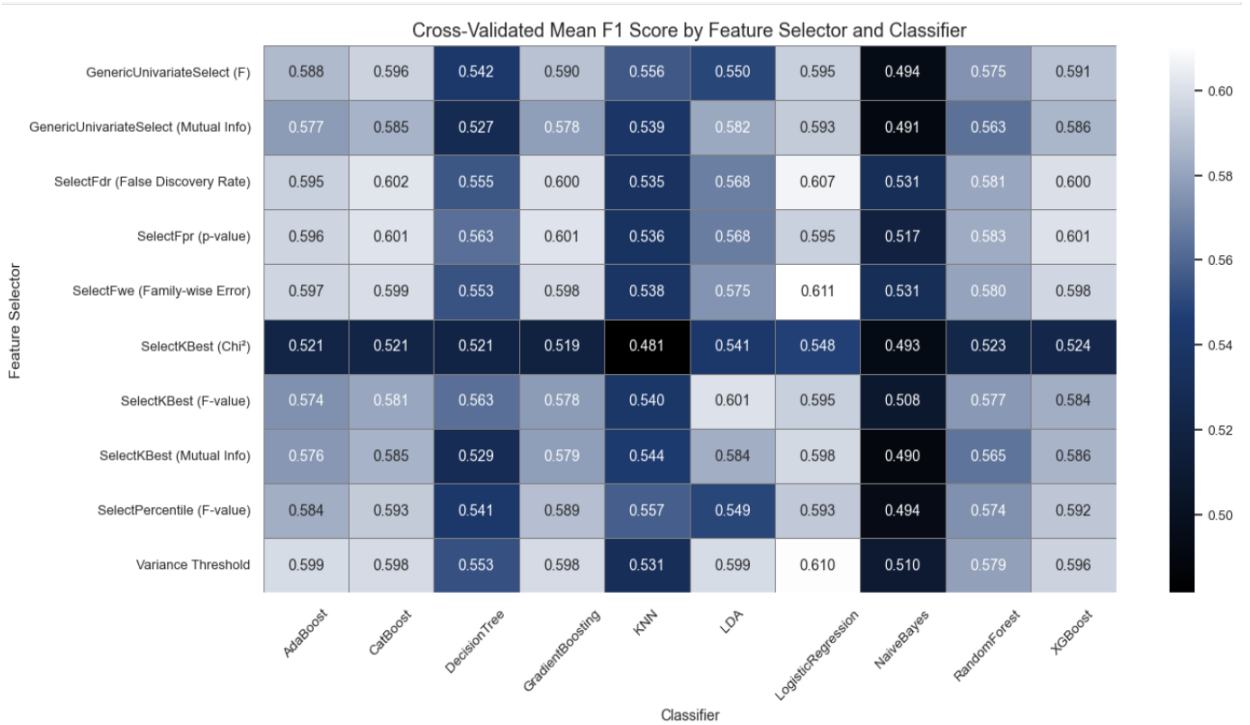


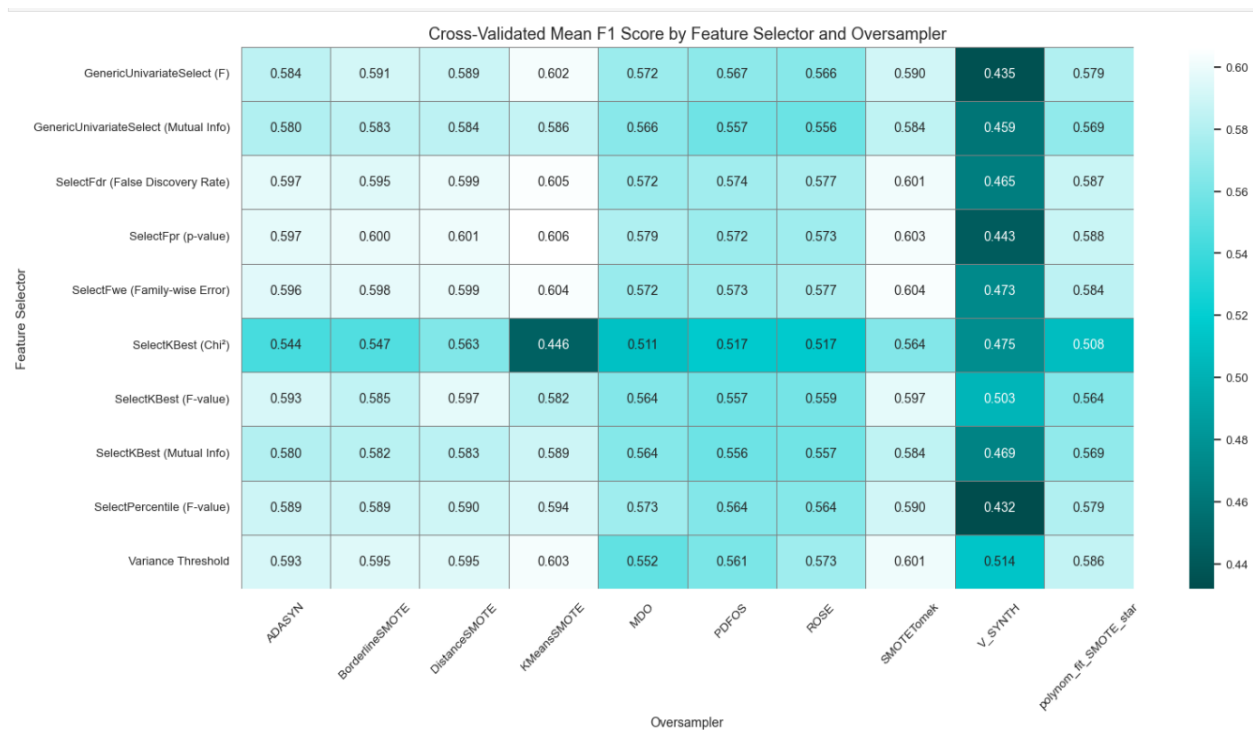
These heatmaps illustrate the **accuracy scores** across all combinations of techniques used in the pipeline, making it easy to compare which setups led to stronger model performance.





These heatmaps illustrate the **F1 Scores** across all combinations of techniques used in the pipeline, making it easy to compare over different techniques.





7. Conclusion

This study presented a comprehensive machine learning pipeline for customer churn prediction using the Telco Customer Churn dataset. By systematically combining data preprocessing, feature selection, oversampling techniques, classifier training, and hyperparameter tuning, we evaluated a total of 1000 unique model configurations.

The primary aim was to improve prediction accuracy by addressing class imbalance and irrelevant features, which are common challenges in real-world churn datasets. The experimental results showed that integrating statistical feature selection methods with advanced oversampling and ensemble learning techniques can significantly boost model accuracy.

The best-performing model achieved an accuracy of 80.91%, utilizing:

- SelectFpr (p-value) for feature selection
- MDO as the oversampling method
- CatBoost as the classifier

These findings demonstrate that no single component guarantees optimal results. Instead, performance depends on the synergy between feature selection, sampling, and classification strategies. The pipeline developed in this study is scalable, modular, and adaptable for other binary classification problems involving imbalanced data.

References

- [1] Reichheld, F. F., & Sasser, W. E. (1990). *Zero Defections: Quality Comes to Services*. Harvard Business Review, 68(5), 105–111.
Available at: hbr.org/1990/09/zero-defections-quality-comes-to-services
- [2] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321–357.
DOI: 10.1613/jair.953
- [3] Maciejewski, T., & Stefanowski, J. (2011). *Local neighbourhood extension of SMOTE for mining imbalanced data*. In IEEE Symposium on Computational Intelligence and Data Mining (CIDM), pp. 104–111.
DOI: 10.1109/CIDM.2011.5949434
- [4] Guyon, I., & Elisseeff, A. (2003). *An Introduction to Variable and Feature Selection*. Journal of Machine Learning Research, 3, 1157–1182.
Available at: jmlr.org/papers/volume3/guyon03a/guyon03a.pdf
- [5] Kohavi, R. (1995). *A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection*. In Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI), Vol. 2, pp. 1137–1145.
Available at: cs.cmu.edu/~kass/ml95/MLhtml/Kohavi.html
- [6] Bergstra, J., & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13(Feb), 281–305.
Available at: jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf
- [7] He, H., & Garcia, E. A. (2009). *Learning from Imbalanced Data*. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284.
DOI: 10.1109/TKDE.2008.239
- [8] Patil, S., & Kulkarni, V. (2020). *Feature Selection and Ensemble Learning for Churn Prediction in Telecom Industry*. In Proceedings of the International Conference on Computational Intelligence and Data Science (ICCIDS), pp. 149–154.
Available at: sciencedirect.com/science/article/pii/S1877050920312722
- [9] Idris, A., Khan, A., & Lee, Y. S. (2012). *Intelligent churn prediction in telecom: Employing mRMR feature selection and RotBoost-based ensemble classification*. Applied Intelligence, 39(3), 659–672.
DOI: 10.1007/s10489-012-0380-3
- [10] Amin, A., Anwar, S., Adnan, A., Nawaz, M., Howard, N., Qadir, J., & Hussain, A. (2016). *Comparing oversampling techniques to handle the class imbalance problem: A customer churn*

prediction case study. IEEE Access, 4, 7940–7957.

DOI: 10.1109/ACCESS.2016.2609424

[11] Burez, J., & Van den Poel, D. (2009). *Handling class imbalance in customer churn prediction*. Expert Systems with Applications, 36(3), 4626–4636.

DOI: 10.1016/j.eswa.2008.05.027

[12] IBM Sample Data Sets. (n.d.). *Telco Customer Churn*. Available at: [kaggle.com/blastchar/telco-customer-churn](https://www.kaggle.com/blastchar/telco-customer-churn)

[13] Batista, G. E. A. P. A., Prati, R. C., & Monard, M. C. (2004). *A study of the behavior of several methods for balancing machine learning training data*. SIGKDD Explorations, 6(1), 20–29.

DOI: 10.1145/1007730.1007735

[14] Liu, H., & Motoda, H. (1998). *Feature Selection for Knowledge Discovery and Data Mining*. Springer Science & Business Media.

[15] Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.

[16] Chandrashekar, G., & Sahin, F. (2014). *A survey on feature selection methods*. Computers & Electrical Engineering, 40(1), 16–28. <https://doi.org/10.1016/j.compeleceng.2013.11.024>

[17] Brown, G., Pocock, A., Zhao, M. J., & Luján, M. (2012). *Conditional Likelihood Maximisation: A Unifying Framework for Information Theoretic Feature Selection*. Journal of Machine Learning Research, 13(Jan), 27–66.

<http://jmlr.org/papers/volume13/brown12a/brown12a.pdf>

[18] Last, F., Douzas, G., & Bacao, F. (2020). *smote-variants: A Python Library for Oversampling Techniques for Imbalanced Classification*. Journal of Open Source Software, 5(46), 2116.

DOI: <https://doi.org/10.21105/joss.02116>

[19] i2tutorials. (n.d.). *What is the difference between Feature Selection and Feature Extraction?* Retrieved from <https://www.i2tutorials.com/what-is-the-difference-between-feature-selection-and-feature-extraction/>

[20] DataAspirant. (n.d.). *How to handle imbalanced data in machine learning*. Retrieved from <https://dataaspirant.com/handle-imbalanced-data-machine-learning/>

[21] Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied Logistic Regression*. Wiley.

[22] Quinlan, J. R. (1986). *Induction of decision trees*. Machine Learning, 1(1), 81–106.

[23] Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32.

[24] Friedman, J. H. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. Annals of Statistics, 29(5), 1189–1232.

- [25] Freund, Y., & Schapire, R. E. (1997). *A decision-theoretic generalization of on-line learning and an application to boosting*. Journal of Computer and System Sciences, 55(1), 119–139.
- [26] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). *CatBoost: unbiased boosting with categorical features*. Advances in Neural Information Processing Systems, 31.
- [27] Rish, I. (2001). *An empirical study of the naive Bayes classifier*. IBM Research Report.
- [28] Cover, T., & Hart, P. (1967). *Nearest neighbor pattern classification*. IEEE Transactions on Information Theory, 13(1), 21–27.
- [29] Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
- [30] Fisher, R. A. (1936). *The use of multiple measurements in taxonomic problems*. Annals of Eugenics, 7(2), 179–188.
- [31] Bergstra, J., & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13(Feb), 281–305.
- [32] Kohavi, R. (1995). *A study of cross-validation and bootstrap for accuracy estimation and model selection*. In IJCAI (Vol. 14, No. 2, pp. 1137–1145).
- [33] ResearchGate. (n.d.). *K-Fold Cross Validation Concept [Figure]*. Retrieved July 24, 2025, from https://www.researchgate.net/figure/K-Fold-Cross-Validation-Concept_fig1_366851964
- [34] Saito, T., & Rehmsmeier, M. (2015). *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. PLoS ONE, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>